SPL-1 PROJECT PROPOSAL

AlgoGenie

See how an algorithm makes its decisions

An offline C++ learning tool with step replay,
solution checks and comparisons on your own inputs.

Md. Sajid Mahmud / BSSE 1738

Software Engineering (IIT), University of Dhaka

A place to test algorithm choices

Students often see the answer without understanding the decisions.

LEARN

Why did this step happen?

Follow decisions at your own pace.
Go back and inspect earlier states.

COMPARE

Which method fits this problem?

Use the same input for both methods.
Check answers and compare work.

For students practising algorithms and teachers explaining them in class.

The menu and program structure

MAIN MENU

1 Learn
2 Compare
3 Load or save a case
4 Export the latest report
0 Exit

Choose problem

Array, graph, coins or activities

↓

Validate and execute

Check input, run the selected solver

↓

Inspect the result

Replay events, verify and save reports

Shared core: custom arrays, graphs, heap, disjoint sets and trace records.

Compare mode: activity selection

Illustrative session: maximise the number of compatible activities

SELECT AND ENTER

Menu 2 / Activity selection
Methods: Greedy + DP

ID	Start	Finish
A	1	3
B	2	5
C	3	4
D	4	6

Menu 4 → comparison.csv

Checks exclude sorting; quadratic DP.

EXPECTED OUTPUT

Method	Selected	Count
Greedy	A, C, D	3
DP	A, C, D	3

Verification: PASS for both
No overlap. Maximum count: 3.

```
algorithm,count,valid,checks
Greedy,3,true,3
DP,3,true,6
```

Learn mode: replay, verify and save

Menu 1 / Activity selection / Greedy / Same four activities

EVENTS THE USER CAN REPLAY

Step	Decision	Selected so far
1	Select A (1, 3)	A
2	Select C (3, 4)	A, C
3	Reject B (2, 5): overlaps	A, C
4	Select D (4, 6)	A, C, D

N: next B: back R: restart Q: return to menu

FINAL CHECK

3 valid activities. No overlap.

SAVE THIS INPUT

Menu 3 → case.txt

Ten solvers across five problems

Input	Planned algorithms	Expected result
Integer array	Merge Sort / Quicksort	Sorted array
Undirected weighted graph	Prim / Kruskal	Minimum spanning tree
Weighted graph + source	Dijkstra / Bellman-Ford	Distances and paths
Coins + target amount	Greedy / DP	Coin selection and count
Activity start/finish times	Earliest-finish Greedy / DP	Maximum compatible set

Dijkstra: nonnegative weights. MST: connected graphs. Coins: unlimited supply.

Implementation and semester milestones

C++17 with handwritten algorithms and data structures

No external libraries. GCC/G++, VS Code and GitHub.
Standard runtime, console and file I/O facilities.

Stage	Working deliverable
Weeks 1–3 / Midterm target	Coin change, merge sort and replay
Weeks 4–7	Activity pair, Quicksort and graph solvers
Weeks 8–11	Verification, benchmarks, TXT and CSV
Weeks 12–14	Testing, documentation and final demo

Final delivery: runnable app, source code, tests and user guide.

Thank you

Questions and suggestions

Md. Sajid Mahmud
BSSE 1738